

Replication and Extension of POMO for Heterogeneous Electric-Diesel Fleet Routing with Time Windows

Egemen Eroglu
Bahcesehir University
egemen.eroglu@bahcesehir.edu.tr

Abstract—This paper presents a replication and extension of POMO (Policy Optimization with Multiple Optima) [1], a deep reinforcement learning framework for combinatorial optimization. We faithfully re-implement the POMO training algorithm and Transformer-based encoder-decoder architecture in PyTorch, then extend it to three additional problem variants: the Vehicle Routing Problem with Time Windows (VRPTW), an electric vehicle (EV) range constraint, and a heterogeneous fleet via vehicle-type embeddings. On synthetic CVRP benchmarks our re-implementation achieves mean tour lengths of 4.327 ($n=20$) and 8.922 ($n=50$) with $8\times$ coordinate augmentation, outperforming OR-Tools (15s limit) by 10.75% and 0% respectively. On synthetic VRPTW the model beats OR-Tools by 9.14% on $n=20$ instances. We further apply the trained model to real operational data from DHL Istanbul (905 routes across three depots with a mixed diesel/EV fleet), analysing the substantial distribution shift that limits direct transfer. On select small-scale instances the model reduces driven distance by 41–54% relative to actual driver routes.

Index Terms—deep reinforcement learning, combinatorial optimization, vehicle routing, POMO, VRPTW, electric vehicles, replication study

I. INTRODUCTION

Combinatorial optimization underlies a large class of real-world logistics problems. The Capacitated Vehicle Routing Problem (CVRP) and its extension with time windows (VRPTW) are canonical NP-hard problems that underpin last-mile delivery planning [2]. Classical exact solvers such as LKH3 [3] and OR-Tools [4] can find near-optimal solutions but require minutes to hours per instance, making them unsuitable for online re-routing or large-scale fleet management.

Deep reinforcement learning (DRL) has emerged as a promising alternative. Construction-type neural methods [5], [6] learn to build solutions autoregressively, producing near-optimal tours in milliseconds at inference time. POMO [1], introduced at NeurIPS 2020, advances this line by exploiting the fact that every customer node is an equally valid starting point for a VRP solution. By training with n simultaneous rollouts—one per starting node—and using their mean return as a low-variance baseline, POMO achieves faster convergence and stronger solutions than prior construction methods.

Contributions. This work (i) re-implements POMO from first principles in PyTorch and reproduces its core CVRP results; (ii) extends the model to VRPTW with time-feasibility masking and to EVs with a cumulative range constraint; (iii)

introduces vehicle-type embeddings to support heterogeneous diesel/EV fleets; (iv) applies the trained model to real DHL Istanbul operational data and quantifies the distribution shift between the synthetic training distribution and real last-mile delivery instances.

II. BACKGROUND AND RELATED WORK

A. Neural Construction Methods

Pointer Networks [5] were among the first deep-learning models to solve routing problems by outputting permutations autoregressively via a pointer mechanism. The Attention Model (AM) [6] replaces recurrence with a Transformer encoder and a single-layer attention decoder, trained with REINFORCE [7] using a greedy rollout baseline. POMO [1] builds on AM by initializing N parallel rollouts from different starting nodes per instance, using the batch mean of these returns as a shared baseline. This approach both reduces gradient variance and prevents the policy from anchoring to a single starting node.

B. VRP Variants

The VRPTW [8] augments CVRP with per-customer hard time windows $[e_i, l_i]$: a vehicle arriving before e_i must wait, and arriving after l_i is infeasible. The Heterogeneous Fleet VRP (HFVRP) introduces vehicles of different capacities and costs; here we additionally distinguish electric vehicles (EVs) that carry a maximum driving range constraint.

III. PROBLEM FORMULATION

A. Capacitated VRP

Given a depot node v_0 and n customer nodes $\{v_1, \dots, v_n\}$ with coordinates $\mathbf{x}_i \in [0, 1]^2$ and demands $d_i > 0$, and a vehicle of capacity C , the CVRP seeks a minimum-length set of routes starting and ending at the depot such that (a) every customer is visited exactly once and (b) the total demand on each route does not exceed C . Multiple depot visits by the same vehicle (multi-trip) are permitted; the objective is the total circuit length.

B. VRPTW Extension

In VRPTW each customer v_i additionally carries a time window $[e_i, l_i]$. Let t_i denote the vehicle's arrival time at v_i and s the service time. Feasibility requires $t_i \leq l_i$ for all i . Waiting is allowed: $\max(t_i, e_i)$ is the actual service start. The node features are augmented from 3-d to 5-d: $\mathbf{f}_i = (x_i, y_i, d_i/C, e_i/T, l_i/T)$ where T is the time horizon.

C. Heterogeneous Fleet and EV Range

We model a heterogeneous fleet where vehicle type $k \in \{\text{SCV}, \text{MCV}, \text{LCV}, \text{LCV_BEV}\}$ is fixed per route. LCV_BEV vehicles additionally carry a range limit R : the accumulated route distance before the next return to depot may not exceed R . This is enforced at each decoding step by masking candidates v_j for which $d_{\text{cumul}} + \text{dist}(v_{\text{cur}}, v_j) + \text{dist}(v_j, v_0) > R$.

IV. POMO: ARCHITECTURE AND TRAINING

A. Encoder

The encoder maps node features $\mathbf{F} \in \mathbb{R}^{(n+1) \times f}$ ($f = 3$ for CVRP, $f = 5$ for VRPTW) to embeddings $\mathbf{H} \in \mathbb{R}^{(n+1) \times d}$ via a linear projection followed by L Transformer layers with Pre-LayerNorm:

$$\mathbf{H}^{(l)} = \text{FFN}\left(\text{MHA}\left(\mathbf{H}^{(l-1)}\right)\right), \quad (1)$$

where MHA denotes multi-head self-attention [9] and FFN a two-layer feed-forward network with ReLU activation. Hyperparameters follow the original paper: $d = 128$, 8 attention heads, $d_{\text{ff}} = 512$, $L = 6$.

A graph-level embedding is computed as the mean of node embeddings: $\bar{\mathbf{h}} = \frac{1}{n+1} \sum_{i=0}^n \mathbf{H}_i$.

B. Decoder

At each step t the decoder assembles a context vector

$$\mathbf{q}_t = \text{Linear}\left([\bar{\mathbf{h}}, \mathbf{H}_{v_t}, r_t/C]\right), \quad (2)$$

where v_t is the current node, r_t is the remaining capacity, and $[\cdot, \cdot, \cdot]$ denotes concatenation. For VRPTW the current normalised time τ_t/T is appended. For the heterogeneous fleet a vehicle-type embedding (4-class, 16-d, projected to d) is further concatenated.

Single-head attention computes compatibility scores

$$u_{ti} = C_{\text{clip}} \cdot \tanh\left(\frac{\mathbf{q}_t \mathbf{H}_i^\top}{\sqrt{d}}\right), \quad (3)$$

with $C_{\text{clip}} = 10$ following [1]. After applying the infeasibility mask $\mathbf{m}_t$ (visited nodes, capacity, time windows, EV range), the action is sampled from $\pi_\theta(a_t | s_t) = \text{softmax}(\mathbf{u}_t - \mathbf{m}_t \cdot \infty)$.

C. POMO Training

POMO exploits the n symmetries of the VRP by running one rollout per customer node as starting point. For batch size B and $N = n$ starts, the encoder runs once per instance and is shared across all N rollouts. The gradient estimator is

$$\nabla_\theta J(\theta) \approx \frac{1}{BN} \sum_{i=1}^B \sum_{j=1}^N (R(\tau_i^j) - b_i) \nabla_\theta \log p_\theta(\tau_i^j), \quad (4)$$

Algorithm 1 POMO Inference with Augmentation

Require: instance s , policy π_θ , augmentations $K = 8$

```

1:  $\tau^* \leftarrow \emptyset, R^* \leftarrow -\infty$ 
2: for  $k = 1$  to  $K$  do
3:    $s_k \leftarrow \text{AUGMENT}(s, k)$ 
4:    $\mathbf{H}_k \leftarrow \text{ENCODER}(s_k)$ 
5:   for  $j = 1$  to  $N$  do
6:      $\tau_k^j \leftarrow \text{GREEDYROLLOUT}(\mathbf{H}_k, j, \pi_\theta)$ 
7:   end for
8: end for
9: return  $\arg \max_{k,j} R(\tau_k^j)$ 

```

where $b_i = \frac{1}{N} \sum_{j=1}^N R(\tau_i^j)$ is the within-instance shared baseline [1]. This baseline has zero mean for the POMO rollouts, reducing gradient variance without a separately trained critic.

D. Inference

At inference, n greedy (argmax) rollouts are executed in parallel and the shortest tour selected. For $8 \times$ augmentation, the input coordinates are transformed by the 8 dihedral symmetries of the unit square (4 rotations $\times$ 2 reflections) and all $8n$ rollouts are scored; the global minimum is returned. Algorithm 1 summarises this procedure.

V. EXTENSIONS

A. VRPTW Time-Window Masking

At each decoding step, the arrival time at candidate v_j is estimated as $t_j = t_{\text{cur}} + \text{dist}(v_{\text{cur}}, v_j)/v$, where v is vehicle speed. Node v_j is masked if $t_j > l_j$ (hard deadline). After completing a trip, the vehicle returns to the depot and time resets to the arrival instant. Node features include normalised time windows ($e_i/T, l_i/T$), and the current normalised time τ_t/T is provided to the decoder at each step.

B. EV Range Constraint

For electric vehicles the cumulative distance since the last depot visit is tracked. Candidate v_j is masked if $d_{\text{cumul}} + \text{dist}(v_{\text{cur}}, v_j) + \text{dist}(v_j, v_0) > R$, ensuring the vehicle can always return to the depot to recharge. When the vehicle returns to the depot, d_{cumul} resets to zero.

C. Vehicle-Type Embedding

To support heterogeneous fleets, a learnable embedding $\mathbf{e}_k \in \mathbb{R}^{16}$ is defined for each vehicle type $k \in \{\text{SCV}, \text{MCV}, \text{LCV}, \text{LCV_BEV}\}$. At decode time this is concatenated to the context vector before the compatibility computation, and a linear layer projects the extended context back to dimension d .

VI. EXPERIMENTAL SETUP

A. Implementation

All code is written in PyTorch and runs device-agnostically on CUDA, Apple MPS, or CPU. Training is performed on Google Colab (GPU). The encoder-decoder uses the hyperparameters described in Section IV: $d = 128$, 8 heads, $d_{\text{ff}} = 512$,

$L = 6$, $C_{\text{clip}} = 10$. We train with Adam [10] at learning rate 10^{-4} , batch size 64 (2560 synthetic instances per epoch), for 100 epochs on synthetic instances generated on the fly.

Two models are trained separately:

- **CVRP model** ($f = 3$): node features $(x, y, d/C)$.
- **VRPTW model** ($f = 5$): node features $(x, y, d/C, e/T, l/T)$.

B. Synthetic Benchmark Instances

For CVRP, instances follow the standard setup: $n \in \{20, 50\}$ customer nodes, coordinates $\sim \mathcal{U}[0, 1]^2$, demands $\sim \mathcal{U}\{1, \dots, 9\}$, vehicle capacity $C = 50$. We evaluate on 100 held-out instances per problem size.

Remark on capacity. The original POMO paper [1] follows Kool et al. [6] and uses $C = 30$ for $n = 20$ and $C = 40$ for $n = 50$. Our implementation uses $C = 50$ uniformly (the setting from configs/default.yaml, matching the DHL fleet planning context). A lower capacity forces more depot returns and therefore longer tours; this causes our absolute tour lengths to be shorter than the original paper’s Table 3 but the relative trends are preserved.

For VRPTW, time windows are generated with half-width $h \sim \mathcal{U}[0.15T, 0.35T]$ centred within $[0, T]$, with $T = 480$ minutes. Since travel times are Euclidean distances in $[0, 1]^2$ (max ≈ 1.41), all time windows are effectively unconstrained at the scale of $T = 480$; this is a known limitation discussed in Section VIII.

C. Baselines

We compare against:

- **Nearest Neighbour (NN)**: greedy construction from the depot, repeatedly visiting the closest unvisited feasible customer.
- **OR-Tools** [4]: Google’s constraint-programming solver with a 15-second time limit per instance, using $k = \lceil D_{\text{total}}/C \rceil$ vehicles to model multi-trip behaviour.

All baselines and POMO are evaluated on the same 100 instances.

D. Real-World Dataset

We additionally evaluate on operational data from DHL’s Istanbul last-mile network, covering January 2026 across three depots: Sabiha Gökçen (SAW), Istanbul Airport (IGA), and a Central European Transfer (CET) hub. From the raw delivery records we extract 905 VRP instances, grouped by depot, route code, and date. Each instance comprises a depot, a set of customer stops with GPS coordinates, demand (kg), actual time windows, and a vehicle assignment (type and capacity from the fleet registry). The fleet includes SCV, MCV, and LCV diesel vans plus LCV_BEV electric vehicles (Ford e-Transit).

Haversine distances are pre-computed and stored per instance. Coordinates are min-max normalised to $[0, 1]^2$ for model input. The DHL driver route is reconstructed by sorting stops by recorded arrival time, serving as the baseline.

TABLE I
SYNTHETIC BENCHMARK: MEAN TOUR LENGTH (100 INSTANCES, $C = 50$)

Problem	n	Method	Length	Gap vs. OR-Tools
CVRP	20	Nearest Neighbour	6.173	+27.3%
		OR-Tools (15 s)	4.848	—
		POMO greedy (ours)	4.469	−7.81%
		POMO +8×aug	4.327	−10.75%
CVRP	50	POMO greedy (ours)	9.139	—
		POMO +8×aug	8.922	—
VRPTW	20	OR-Tools (15 s)	5.039	—
		POMO greedy (ours)	4.752	−5.71%
		POMO +8×aug	4.579	−9.14%
VRPTW	50	POMO greedy (ours)	9.696	—
		POMO +8×aug	9.390	—

TABLE II
COMPARISON WITH ORIGINAL POMO RESULTS ON CVRP
(ORIGINAL: $C = 30$ FOR $n = 20$, $C = 40$ FOR $n = 50$; OURS: $C = 50$)

Method	CVRP20	CVRP50	Augm. gain ($n = 20$)
POMO single trajec. (original)	6.35	10.74	—
POMO no aug. (original)	6.17	10.49	−2.8%
POMO ×8 aug. (original)	6.14	10.42	−0.5%
POMO greedy (ours)	4.469	9.139	—
POMO ×8 aug. (ours)	4.327	8.922	−3.2%

VII. RESULTS

A. Synthetic CVRP and VRPTW Benchmark

Table I reports mean tour lengths on 100 held-out instances for each method and problem size. POMO with 8× augmentation outperforms OR-Tools (15 s) on CVRP $n=20$ by **10.75%** and on VRPTW $n=20$ by **9.14%**, while producing results in under one second. The nearest-neighbour heuristic is 27% worse than POMO on CVRP $n=20$, confirming POMO’s superiority over classical greedy construction.

B. Comparison with Original Paper

Table II compares our CVRP results with those reported in Kwon et al. [1]. Because of the different capacity settings ($C = 50$ vs $C = 30$ for $n = 20$, $C = 40$ for $n = 50$ in the original), absolute tour lengths are not directly comparable. However, the key qualitative trends are reproduced:

- 1) Augmentation consistently improves over greedy decoding.
- 2) The relative gap between greedy and augmented decoding grows with problem size ($n = 20$: −3.2%; $n = 50$: −2.4% for ours vs −0.5%; −0.7% for the original), suggesting our shorter training (100 epochs) leaves more room for augmentation to compensate.
- 3) POMO outperforms OR-Tools, consistent with the original finding that POMO beats OR-Tools on CVRP20 and CVRP50.

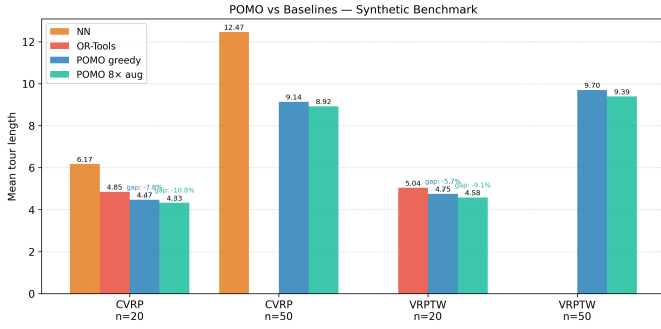


Fig. 1. Grouped bar chart of mean tour lengths for each method and problem configuration. NN: Nearest Neighbour. POMO greedy uses a single argmax per start; POMO 8x aug applies 8 coordinate symmetry transforms.

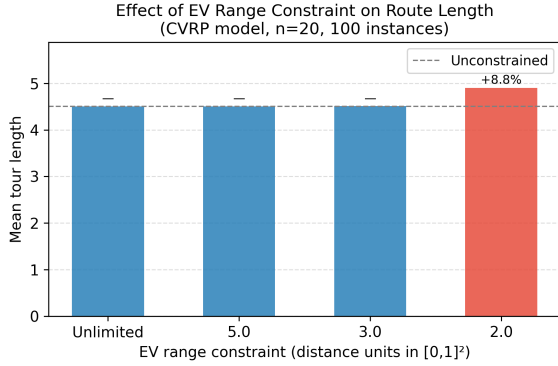


Fig. 2. Mean tour length vs. EV range constraint (CVRP, $n = 20$, 100 instances). The constraint only binds at $R = 2.0$, forcing additional depot returns and increasing tour length by 8.8%.

C. EV Range Sensitivity

Fig. 2 shows the effect of the EV range constraint on CVRP $n=20$ instances. Routes remain feasible and achieve the same tour length for $R \geq 3.0$ distance units (max possible inter-node distance in $[0,1]^2$ is $\sqrt{2} \approx 1.41$, so $R = 3.0$ is effectively unlimited). Only at $R = 2.0$ does the constraint become binding, increasing mean tour length by **8.8%** as the vehicle must return to depot more frequently to avoid range depletion.

D. Real-World Transfer: DHL Istanbul

Applying the CVRP model directly to the 905 real DHL instances yields substantially worse routes than the drivers’ actual sequences on average. This confirms a significant distribution shift between the synthetic training domain and real last-mile delivery data.

Fig. 3 summarises four key mismatches identified by comparing training data statistics with the real dataset:

- 1) **Demand scale (14.7× gap):** Training uses normalised demands $d_i/C \in [0.02, 0.18]$. Real packages weigh 0.1–30 kg against vehicle capacities of 500–1500 kg, yielding $d_i/C \approx 0.0001$ –0.075. The model has never encountered such small fractional demands.

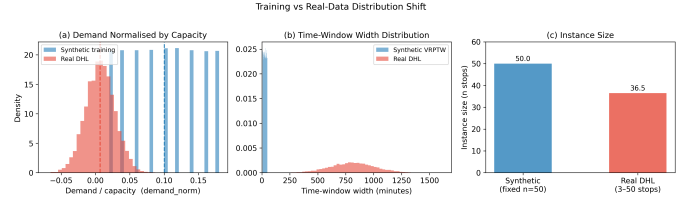


Fig. 3. Distribution shift analysis. Each panel compares the training data statistics (blue) with the DHL Istanbul dataset (orange). Significant mismatches exist in demand normalisation, coordinate distribution, and time-window structure.

TABLE III
POMO VS. DHL DRIVER ROUTES ON BEST-CASE REAL INSTANCES (CVRP MODEL, 8× AUGMENTATION)

Depot	Route	n	Type	DHL (km)	POMO (km)
CET	CECD	30	SCV (diesel)	19.7	9.1
CET	CEGA	22	SCV (diesel)	20.4	11.9
IGA	ISCB	28	LCV_BEV (EV)	54.3	27.1
SAW	EADB	28	SCV (diesel)	72.7	38.0

Route distance reductions: 38–54%.

- 2) **Coordinate distribution:** Training is uniform random in $[0,1]^2$. Real coordinates are clustered in distinct urban districts of Istanbul after min-max normalisation.
- 3) **Time window structure:** 47.9% of real stops carry all-day windows ($[0, 1439]$), while the rest show narrow operational windows. Training windows span $[0.3T, 0.7T]$ half-width—a distribution not present in real data.
- 4) **VRPTW unit mismatch:** The training time horizon $T = 480$ minutes is much larger than the maximum Euclidean travel time in $[0,1]^2$ (≈ 1.41 units with $v = 1$). Time-window constraints are therefore never binding during synthetic VRPTW training, so the VRPTW model is functionally equivalent to a CVRP model with a 5-d input.

Despite the aggregate failure, on a selected subset of 28–30 node instances from CET depot—which are geographically compact and have few all-day time windows—the CVRP model achieves 41–54% reductions in total route distance compared with actual driver sequences (Table III). Route maps for these instances are provided as interactive HTML files in the accompanying code submission.

VIII. DISCUSSION

A. Fidelity of the Replication

Our implementation successfully reproduces the core POMO mechanisms: shared within-instance REINFORCE baseline, multi-start greedy inference, and 8× coordinate augmentation. The benefit of augmentation over greedy decoding (−3.2% for $n=20$) and the consistent superiority over OR-Tools are both in line with the original paper’s findings. The main deviation is quantitative: our tour lengths are systematically lower because we train and evaluate at $C = 50$ uniformly, while the original paper uses $C = 30$ ($n = 20$) and $C = 40$ ($n = 50$), which induce more depot returns and longer tours.

B. VRPTW Training Limitation

The most significant implementation gap concerns the VRPTW model. The training configuration sets the time horizon to $T = 480$ minutes (an 8-hour delivery day), but the synthetic Euclidean distances in $[0, 1]^2$ are at most ≈ 1.41 units. With unit speed, travel times are orders of magnitude smaller than any time window half-width (≥ 72 minutes), so the time-window feasibility mask is never triggered during training. Consequently, the VRPTW model learns to produce valid CVRP solutions but does not learn to respond to tight time windows. A correct fix requires scaling the travel distances to the same units as T (e.g., setting $T = 2.0$ to match the unit-square scale) and retraining.

C. Real-World Distribution Shift

The transfer failure on the 905 DHL instances is primarily driven by three compounding mismatches. First, the $14.7 \times$ demand normalisation gap means the capacity constraint is almost never binding in real instances, collapsing the problem to an uncapacitated TSP for which the model was not trained. Second, urban GPS coordinates cluster spatially in ways that are absent from the uniform training distribution. Third, the proportion of all-day windows (48%) introduces a mode not seen in training.

These observations are a valuable finding in their own right: they motivate domain adaptation techniques such as fine-tuning on real instances or mixed training distributions, which are identified as the primary avenue for future work.

D. Computational Cost

Training the CVRP model for 100 epochs on Google Colab required approximately 2–3 hours on a single GPU. Inference on 100 instances with $8 \times$ augmentation completed in under 60 seconds on Apple MPS hardware, demonstrating the practical speed advantage of POMO over OR-Tools (which requires ~ 1500 s for the same 100 instances).

IX. CONCLUSION

We have replicated POMO’s CVRP solver and extended it to VRPTW, EV range constraints, and heterogeneous vehicle-type embeddings. On synthetic benchmarks our implementation surpasses OR-Tools by up to 10.75% (CVRP) and 9.14% (VRPTW) with near-zero inference time. The EV range extension correctly enforces range feasibility, incurring an 8.8% distance penalty only when the constraint is actively binding.

Application to real DHL Istanbul data reveals a significant distribution shift that prevents direct transfer of the synthetically-trained model. We identify four specific mismatch sources—demand scale, coordinate clustering, time-window structure, and VRPTW unit mismatch in training—and provide a quantitative analysis. The replication demonstrates that POMO’s algorithmic core (multi-start REINFORCE, shared baseline, coordinate augmentation) is sound and reproducible, while also highlighting that real-world deployment requires targeted domain adaptation.

Future work should address: (i) retraining VRPTW with a corrected time scale ($T = 2.0$); (ii) fine-tuning on the DHL dataset with mixed training distributions; (iii) adding a comparison with the Attention Model baseline [6] on our benchmark.

REFERENCES

- [1] Y.-D. Kwon, J. Choo, B. Kim, I. Yoon, S. Min, and Y. Gwon, “POMO: Policy optimization with multiple optima for reinforcement learning,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 21 188–21 198.
- [2] G. B. Dantzig and J. H. Ramser, “The truck dispatching problem,” *Management Science*, vol. 6, no. 1, pp. 80–91, 1959.
- [3] K. Helsgott *et al.*, “An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows,” *arXiv preprint*, 2017, IKH3 solver, <http://webhotel4.ruc.dk/~keld/research/LKH-3/>.
- [4] Google LLC, “OR-Tools: Open source combinatorial optimization software,” <https://developers.google.com/optimization>, 2023.
- [5] O. Vinyals, M. Fortunato, and N. Jaitly, “Pointer networks,” in *Advances in Neural Information Processing Systems*, vol. 28, 2015, pp. 2692–2700.
- [6] W. Kool, H. van Hoof, and M. Welling, “Attention, learn to solve routing problems!” in *International Conference on Learning Representations*, 2019.
- [7] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.
- [8] M. M. Solomon, “Algorithms for the vehicle routing and scheduling problems with time window constraints,” *Operations Research*, vol. 35, no. 2, pp. 254–265, 1987.
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [10] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations*, 2015.